

Assignment-2

Name: Jawad Ahmad - LIF22BSCSO316

Task-1

• Purpose of Language:

SaraikiScript is a Saraiki inspired programming language. It uses authentic Saraiki vocabulary to make programming accessible to Saraiki speakers from southern Punjab, Pakistan.

• Style of Syntax:

SaraikiScript follows a C-style imperative syntax with following characteristics:

- Uses curly braces `{ }` for code blocks.
- Each statement terminate with a semi-colon.
- Explicit data type declarations
- Distinguishes between uppercases and lowercase identifiers.
- Single-line `//` and multi-line `/* */` comments.

• Reason for Choosing Keywords:

- Keywords map directly to Saraiki words
- Reduces mental translation overhead between English and Saraiki
- Promotes regional language in technical domains

•

• Keywords, Operators and Punctuations:

Keyword	English Equivalent	Meaning
je	if	conditional Statement
bia	else	Alternative condition
jein-tak	while	Loop iteration
wastey	for	iteration loop
wapas	return	Return from function

Operators	Type	Description
=	Assignment	Assign value
= =	Relational	Equality comparison
+	Arithmetic	Addition operator
Symbol	Name	Purpose
;	Semi-colon	Statement terminator
{ }	Curly braces	Code block delimiters

Task-2

1. Program Structure:

Program → Statement List

Statement List → Statement | Statement Statement List

Statement → Declaration | Assignment | conditional |
Loop | IO-Statement.

2. Variable Declaration:

Declaration $\rightarrow$ DataType Identifier ;

Declaration $\rightarrow$ DataType Identifier = Expression ;

DataType $\rightarrow$ int | float | char | string | bool | void

3. Assignment Statement:

Assignment $\rightarrow$ Identifier = Expression ;

Expression $\rightarrow$ Identifier | Number | Expression Operator Expression

Operator $\rightarrow$ + | - | * | / | %

Number $\rightarrow$ Integer | Float

4. Conditional Statement:

conditional $\rightarrow$ je (condition) { StatementList }

conditional $\rightarrow$ je (condition) { StatementList } bia { StatementList }

condition $\rightarrow$ Expression Rel_OP Expression

Rel_OP $\rightarrow$ == | != | > | < | >= | <=

5. Loop Statement:

Loop $\rightarrow$ jein-tak (condition) { StatementList }

Loop $\rightarrow$ wastey (Assignment condition ; Assignment)
{ StatementList }

6. Input/output Statement:

IO-Statement $\rightarrow$ das << Expression;

IO-statement $\rightarrow$ vikha << Expression;

Io-Statement $\rightarrow$ Identifier >> identifier;

Task - 3

Rule 1 : Program $\rightarrow$ Statement List

Rule 2 : StatementList $\rightarrow$ Statement StatementList

Rule 3 : Statement List $\rightarrow$ Statement

Rule 4 : Statement $\rightarrow$ Declaration

Rule 5 : Statement $\rightarrow$ Assignment

Rule 6 : Statement $\rightarrow$ conditional

Rule 7 : Declaration $\rightarrow$ DataType Identifier;

Rule 8 : Assignment $\rightarrow$ identifier = Expression;

Rule 9 : Expression $\rightarrow$ Identifier

Rule 10 : Expression $\rightarrow$ INTEGER

Rule 11 : conditional $\rightarrow$ je (condition) {StatementList}

Rule 12 : condition $\rightarrow$ Expression Rel-OP Expression

Rule 13 : Rel-OP $\rightarrow$ ==

Rule 14 : DataType $\rightarrow$ int

Rule 15 : Loop $\rightarrow$ jein-tak (condition) {StatementList}

⇒ Example parse using the Rules:

int x; x = 5;

⇒ Derivation:

Program → StatementList (Rule 1)

StatementList → Statement StatementList (Rule 2)

Statement → Declaration (Rule 4)

→ int x; StatementList (Rule 7, 14)

→ int x; Statement (Rule 3)

→ int x; Assignment (Rule 5)

→ int x; x = Expression; (Rule 8)

→ int x; x = 5; (Rule 10)

Task-4

Expression → Identifier INTEGER	First { Identifier, INTEGER }	Follow { ; , == }
-----------------------------------	-------------------------------------	----------------------

Statement → Declaration Assignment conditional	First { int, float, char, string, bool, void, Identifier, je }	Follow { int, float, char, string, bool, void, Identifier, je, {, \$ }
--	---	--

→ Both the statements are taken from Production Rules.

Task-5

No

~~Yes~~, grammar is $\downarrow$ ambiguous.
Not

It is because:

→ No recursive Productions

→ No Operator Precedence issues

→ Each rule produces distinct terminals

but if we add nested conditionals without restrictions:

conditional $\rightarrow$ je (condition) { statementList }

conditional $\rightarrow$ ~~je (condition)~~ ~~Expression~~ ~~Rel OP~~ ~~Expression~~ { statementList } bia { statementList }

→ Dangling Else Problem:

je (x==1) { je (y==2) { z=3; } bia { z=4; } }

The 'bia' could belong to First 'je' (outer if) or second 'je' (inner if)

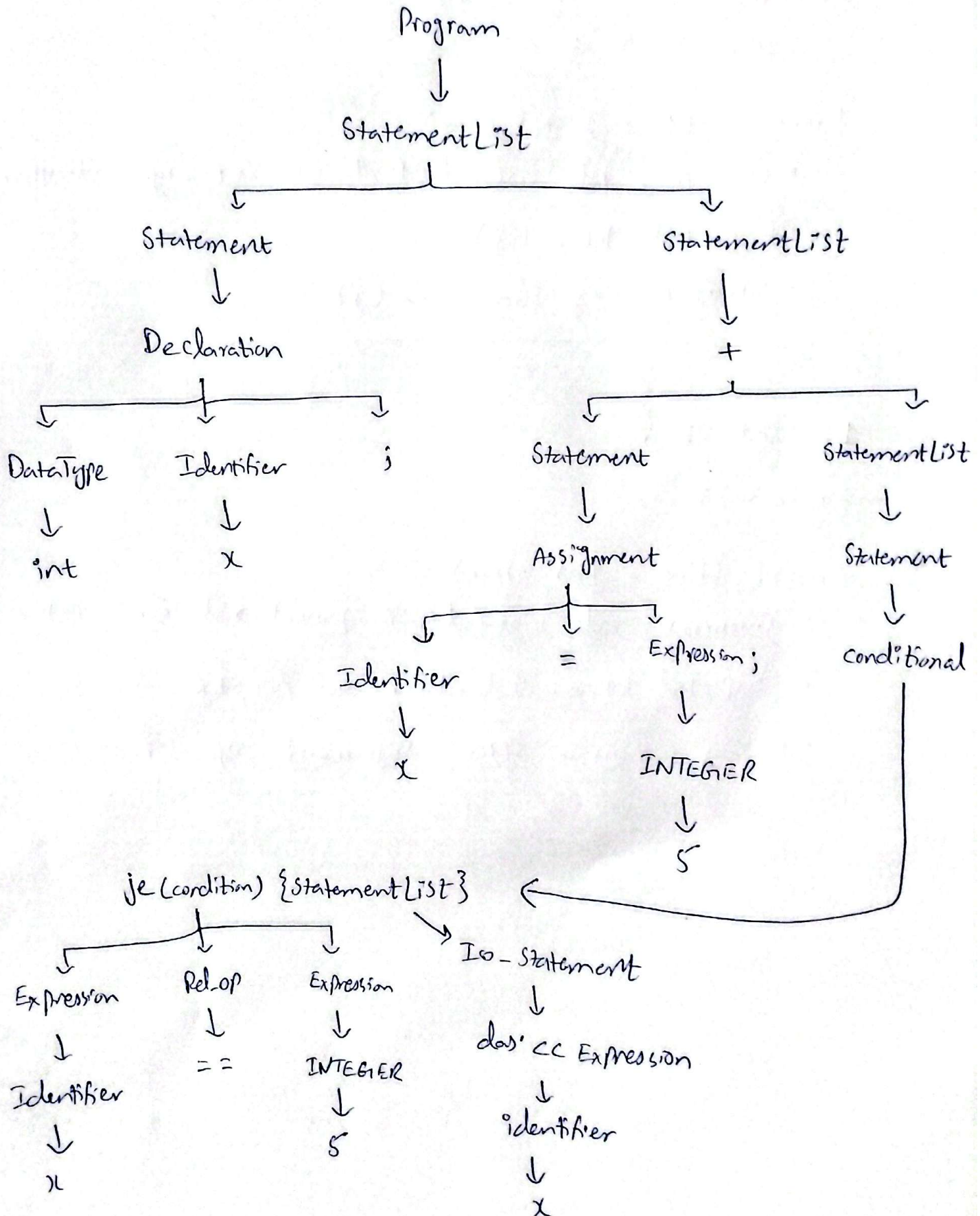
→ Resolution:

Match 'bia' to the nearest unmatched 'je':

Parser Rule: Associate 'bia' with the closest preceding 'je' that doesn't already have 'bia'

Task - 6

int x ; x = 5 ; je (x == 5) { das << x ; }



Task - 7

~~int x ; x = 5 ;~~

1. int x

2. ~~int~~ x = 5 ;

Errors: (Line 1 has error)

- Grammar rule violated: Declaration $\rightarrow$ DataType Identifier ;
 - Expected token (;)
 - Error: Missing Semi-colon (;)
-

1. int z ;

2. z = ;

Errors: (Line 2 has error)

- Grammar rule violated : Assignment $\rightarrow$ identifier = Expression ;
- Expected token: Identifier or INTEGER
- Missing Expression after assignment operator.